

STUDY GUIDE · FROM ZERO TO RUNNING IT YOURSELF

EKS DevSecOps Platform — Study Guide

Every tool explained for beginners: AWS regions and IAM, VPC networking, Terraform, Amazon EKS, a secure container image for a React app, Trivy scanning, GitHub Actions CI, OIDC and least privilege, the deploy pipeline, the AWS Load Balancer Controller, Kubernetes workload security, zero-downtime deployments and a verified teardown, plus nine hands-on labs, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-eks-devsecops>

Contents

- 0. The big picture
- 1. AWS basics for this project
- 2. VPC networking
- 3. Terraform
- 4. Amazon EKS
- 5. Containers for the app
- 6. Trivy and scanning
- 7. GitHub Actions CI
- 8. OIDC and least privilege
- 9. The deploy pipeline
- 10. Load Balancer Controller and Ingress
- 11. Kubernetes workload security
- 12. Zero-downtime deployments
- 13. Teardown and cost
- 14. Hands-on labs

Glossary

Interview questions

Each chapter: what the tool is, why this project uses it, how it works, where it is integrated, commands to try, common mistakes, and questions with answers.

0. The big picture

The story in one paragraph

A small website (a profile card written in React) has to go from a developer's laptop to the internet. It should run on **Amazon EKS** (Kubernetes managed by AWS), every change should be **checked and deployed automatically**, and nobody should be able to break in easily. At the end, everything is **deleted**, and the deletion is **proven**. This project does exactly that, and everything in it was really built, attacked and deleted on an AWS account.

An everyday analogy

Think of a restaurant kitchen:

Kitchen	This project
The building, with a back door for deliveries and a front door for guests	the VPC : private subnets (kitchen) and public subnets (front door)
The kitchen staff and stoves	the EKS cluster and its worker nodes
The recipe book everyone follows	Terraform : the infrastructure written down as code
Health inspection before a dish leaves the kitchen	Trivy scanning and the CI checks
A staff badge that only opens one door, for one shift	OIDC : short-lived AWS access for the pipeline, for one namespace
The waiter who brings food to guests	the Application Load Balancer , created from a Kubernetes Ingress
Kitchen rules: no fire in the cold room, staff only in their area	Pod Security and NetworkPolicies
Closing time: everything off, doors locked, checklist signed	teardown and its verification script

What happens when a developer pushes code

```
git push → GitHub Actions
  CI (every push and pull request): secret scan, tests, dependency audit, Dockerfile lint,
                                     image scan, Terraform + Kubernetes checks, pipeline checks
  CD (push to main): job 1 build + scan (no cloud access)
                    job 2 get temporary AWS access (OIDC) → push to ECR → sign → deploy → test
AWS (Frankfurt, eu-central-1)
  VPC with 2 availability zones
    public subnets: load balancer, NAT gateway
    private subnets: EKS worker nodes (no public IPs) running 2 copies of the app
```

What was measured

	Result
Build everything with Terraform	72 resources in 12 min 30 s
<code>git push</code> to live website	3 min 37 s
First image scan	blocked : a real HIGH vulnerability in the base image, then fixed
Privileged pod, outbound traffic, stealing node credentials	all refused or blocked
Another pod reaching the app directly	worked at first, then fixed and blocked
Requests during a deployment	3 of 280 failed, then fixed: 401 of 401 succeeded
Delete everything	72 resources destroyed, verification clean, 62 minutes on AWS in total

Two attacks found real weaknesses. That is normal and good: the point of testing is to find them before an attacker does. You will learn both fixes in chapters 11 and 12.

The tools, one line each

Tool	Job in this project	Chapter
AWS (IAM, regions, tags)	the cloud account and its rules	1
Amazon VPC	the private network	2
Terraform	builds and deletes all AWS resources from code	3
Amazon EKS	runs Kubernetes for us	4
React, Vite, Docker, nginx	the app and its container image	5
Trivy	finds vulnerabilities and misconfigurations	6
GitHub Actions	runs the checks and the deployment	7, 9
OIDC, IAM roles, EKS access entries	access without stored keys, least privilege	8
Amazon ECR, cosign	stores and signs the images	9
AWS Load Balancer Controller	turns an Ingress into a load balancer	10
Pod Security, NetworkPolicy	rules for what pods may do	11
Readiness gates, preStop	deployments without errors	12
<code>scripts/teardown.sh</code>	deletes everything and proves it	13

Check yourself

1. Where do the worker nodes run, and can someone on the internet reach them?
2. What does the pipeline need to reach AWS, and what is *not* stored anywhere?
3. Why is it good news that two attacks worked at first?

Answers

▼ Answers

1. In the private subnets of the VPC, without public IP addresses. Only the load balancer in the public subnets is reachable from the internet.
2. A short-lived token from GitHub (OIDC) that AWS exchanges for temporary credentials. No AWS access key is stored in GitHub.
3. Because they were found and fixed in a test, with measurements, instead of being found by an attacker in production.

1. AWS basics for this project

What is it?

Amazon Web Services (AWS) rents out computers, networks and managed services by the hour. A few ideas are enough for this project:

Idea	Meaning
Account	your container for all AWS resources and the bill; identified by a 12-digit number
Region	a geographic area with its own data centers, for example <code>eu-central-1</code> (Frankfurt)
Availability zone (AZ)	one or more separate data centers inside a region; a region has at least three
IAM	Identity and Access Management: <i>who</i> (users, roles) may do <i>what</i> (policies) on <i>which</i> resources
IAM role	an identity without a password that someone or something can assume to get temporary credentials
Tag	a key/value label on a resource, for example <code>Project=eks-devsecops</code>

Why this project cares

- **One region only.** The project was built in an account that other people use. Working in exactly one region (Frankfurt) keeps the project away from everything else, and makes cleanup easy to verify.
- **Two availability zones.** If one data center fails, the app keeps running in the other one.
- **IAM is global.** Unlike most resources, IAM roles and policies are not tied to a region. EKS cannot work without IAM roles, so the project creates a few **uniquely named** ones (all starting with `eks-devsecops`) and deletes them at the end.
- **Tags on everything.** Every resource gets `Project=eks-devsecops`, `Owner=sufyan`, `ManagedBy=terraform`. At the end, a script searches for these tags to prove nothing was left behind.

How it works

Regions and AZs

```
eu-central-1 (Frankfurt)
├─ eu-central-1a - private subnet 10.42.0.0/20, public subnet 10.42.48.0/24
└─ eu-central-1b - private subnet 10.42.16.0/20, public subnet 10.42.49.0/24
```

Terraform picks the first two available zones of the region

(`slice(data.aws_availability_zones.available.names, 0, 2)` in `terraform/vpc.tf`).

IAM: principals, policies, roles

- A **principal** is someone or something that makes requests: an IAM user, a role, an AWS service.
- A **policy** is a JSON document: *allow these actions on these resources, under these conditions*.

- A **trust policy** is attached to a role and says *who may assume it*. In chapter 8 the trust policy says: "only GitHub Actions jobs of this repository, in its `production` environment".

Cost

You pay for most things **per hour** while they exist. This lab's main costs in Frankfurt, at on-demand prices:

Item	About
EKS control plane	\$0.10 / hour
2 × t3.medium worker nodes	\$0.10 / hour
NAT gateway	\$0.05 / hour + data
Load balancer, public IPv4 addresses, logs, KMS key	~\$0.05 / hour

That is **roughly \$0.30 per hour**. The recorded build ran 62 minutes: an estimated \$0.35. A forgotten lab costs about \$7 per day, so deleting it matters (chapter 13).

Where it is configured

`terraform/main.tf` applies the tags to every resource the AWS provider creates:

```
locals {
  name = "eks-devsecops"
  tags = {
    Project    = "eks-devsecops"
    Owner      = "sufyan"
    ManagedBy  = "terraform"
    Repository = "github.com/${var.github_repository}"
  }
}

provider "aws" {
  region                = var.region
  allowed_account_ids = [var.allowed_account_id]
  default_tags {
    tags = local.tags
  }
}
```

`terraform/variables.tf` refuses any region but Frankfurt:

```
variable "region" {
  default = "eu-central-1"
  validation {
    condition     = var.region == "eu-central-1"
    error_message = "This project only runs in eu-central-1 (Frankfurt)."
```

Try it

With the AWS CLI configured for **your own** account:

```
aws sts get-caller-identity --query Arn          # who am I?
aws ec2 describe-availability-zones --region eu-central-1 --query 'AvailabilityZones[].ZoneName'
aws ec2 describe-vpcs --region eu-central-1 --query 'length(Vpcs)' # take an inventory before you
start
```

Write the inventory down: after the teardown you compare against it.

Common mistakes

- Running commands with the wrong profile or account. Terraform's `allowed_account_ids` guard prevents that.
- Forgetting that IAM is global: deleting "everything in the region" does not delete IAM roles.
- Leaving a lab running overnight.
- Sharing screenshots that show the account ID in ARNs or registry hostnames.

Check yourself

1. What is the difference between a region and an availability zone?
2. Why does the project use two availability zones?
3. Which part of this project is *not* regional, and how is it kept separate from other people's resources?

Answers

▼ Answers

1. A region is a geographic area (Frankfurt); an availability zone is an isolated data center (or group of them) inside a region.
2. So that the loss of one data center does not take the app down: one replica runs in each zone.
3. IAM (roles and policies). The project gives every IAM name the prefix `eks-devsecops` and tags them, and the teardown verification checks both.

2. VPC networking

What is it?

A **VPC** (Virtual Private Cloud) is your own private network inside AWS. You choose its address range and split it into **subnets**.

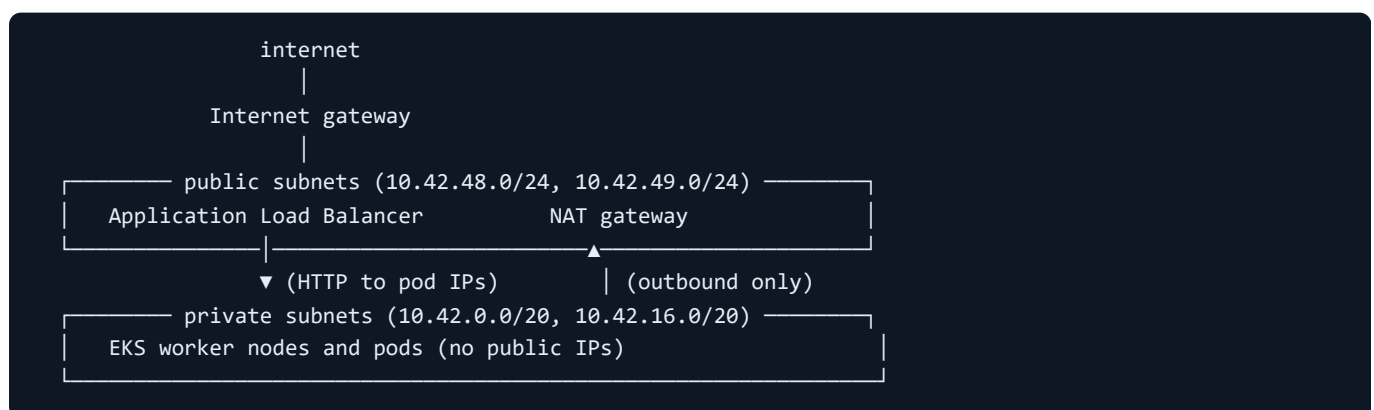
Part	Job
CIDR block	the address range, written like <code>10.42.0.0/16</code> (65,536 addresses starting at 10.42.0.0)
Subnet	a slice of the VPC inside one availability zone
Internet gateway (IGW)	the VPC's door to the internet
Public subnet	a subnet whose route table sends internet traffic to the IGW
Private subnet	a subnet with no route from the internet
NAT gateway	lets private machines <i>start</i> connections to the internet (to download images), while nothing can connect <i>in</i>
Route table	the rules "traffic for X goes to Y" for a subnet
Security group (SG)	a firewall around a network interface: which traffic may come in and go out
VPC flow logs	records of every connection: source, destination, port, accepted or rejected

Why this project uses it

The worker nodes that run the app should **not** be on the internet. Only the load balancer should be. So:

- **Private subnets** hold the nodes and the pods. They have no public IP addresses.
- **Public subnets** hold only the Application Load Balancer and the NAT gateway.
- The **NAT gateway** lets the nodes pull container images and updates.

How it works



- The public subnets' route table: `0.0.0.0/0 → internet gateway`.
- The private subnets' route table: `0.0.0.0/0 → NAT gateway`.

- **One NAT gateway** keeps the lab cheap. In production you use one per availability zone, so losing a zone does not cut off the other zone's internet access.

Subnet tags for load balancers

The AWS Load Balancer Controller (chapter 10) needs to know where to put load balancers. It looks for tags: -

`kubernetes.io/role/elb = 1` on public subnets (internet-facing load balancers) -

`kubernetes.io/role/internal-elb = 1` on private subnets (internal load balancers)

An important EKS detail for later

With the **VPC CNI** (the network plugin of EKS), **Pods get IP addresses from the VPC's private subnets**. In this project the two app pods had the IPs 10.42.8.223 and 10.42.18.155. That matters in chapter 11: a firewall rule that says "allow the VPC" also allows every pod in the cluster.

Where it is configured

`terraform/vpc.tf` (community module `terraform-aws-modules/vpc/aws`, version 6.7.3):

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "6.7.3"

  name = local.name
  cidr = "10.42.0.0/16"
  azs = slice(data.aws_availability_zones.available.names, 0, 2)

  private_subnets = ["10.42.0.0/20", "10.42.16.0/20"]
  public_subnets = ["10.42.48.0/24", "10.42.49.0/24"]

  enable_nat_gateway = true
  single_nat_gateway = true

  public_subnet_tags = { "kubernetes.io/role/elb" = 1 }
  private_subnet_tags = { "kubernetes.io/role/internal-elb" = 1 }

  manage_default_security_group = true
  default_security_group_ingress = []
  default_security_group_egress = []

  enable_flow_log = true
  create_flow_log_cloudwatch_log_group = true
  create_flow_log_cloudwatch_iam_role = true
  flow_log_max_aggregation_interval = 60
  flow_log_cloudwatch_log_group_retention_in_days = 7
}
```

Two security habits are in there: - The VPC's **default security group allows nothing**, so nothing accidentally attached to it is open. - **Flow logs** record connections every 60 seconds, kept for 7 days, which you will want during an investigation.

Try it

After `terraform apply` (lab 6):

```
aws ec2 describe-subnets --filters Name=tag:Project,Values=eks-devsecops \
--query 'Subnets[][CidrBlock,AvailabilityZone,MapPublicIpOnLaunch]' --output table
kubectl get nodes -o wide          # EXTERNAL-IP must be <none>
```

Common mistakes

- Putting worker nodes in public subnets "to make it work". Use a NAT gateway instead.
- Forgetting the subnet tags: the load balancer controller cannot find a subnet and the Ingress stays without an address.
- Choosing a CIDR that overlaps with a network you may connect to later (office VPN, another VPC).
- Assuming a security group or "the VPC CIDR" separates pods: on EKS, pod IPs are VPC IPs.

Check yourself

1. What is the difference between an internet gateway and a NAT gateway?
2. Why do the private subnets need a NAT gateway at all?
3. What would break if the public subnets had no `kubernetes.io/role/elb` tag?

Answers

▼ Answers

1. The internet gateway allows traffic in both directions for resources with public IPs. A NAT gateway only lets private resources start outgoing connections; nothing from the internet can connect in through it.
2. The nodes must download container images (ECR), reach the EKS API and other AWS services, and fetch OS updates.
3. The AWS Load Balancer Controller would not find a subnet for an internet-facing load balancer, so the Ingress would get no address.

3. Terraform

What is it?

Terraform builds infrastructure from text files. You describe *what* should exist (a VPC, a cluster, a role); Terraform works out *how* to create, change or delete it.

Idea	Meaning
Provider	a plugin that talks to one API: <code>hashicorp/aws</code> , <code>hashicorp/helm</code> , <code>hashicorp/kubernetes</code>
Resource	one thing Terraform manages, e.g. <code>aws_ecr_repository.app</code>
Data source	something Terraform only reads , e.g. the existing GitHub OIDC provider
Module	a reusable package of resources, e.g. <code>terraform-aws-modules/eks/aws</code>
State	Terraform's record of what it created (<code>terraform.tfstate</code>)
plan	a preview: what will be added, changed and destroyed
apply	do it
destroy	delete everything in the state

Why this project uses it

- **Reviewable:** infrastructure changes are pull requests, not clicks.
- **Repeatable:** every student gets the same platform.
- **Destroyable:** one command deletes all 72 resources; you cannot forget one in a console.

Alternatives: AWS CloudFormation or CDK (AWS only), Pulumi (general-purpose languages), `eksctl` (EKS only, quick but less control), OpenTofu (an open-source fork of Terraform).

How it works

1. `terraform init` downloads providers and modules (pinned versions, recorded in `.terraform.lock.hcl`).
2. `terraform plan -out=tfplan` compares your files with the state and the real world, and saves the plan.
3. **Read the plan.** The recorded plan ended with **Plan: 73 to add, 0 to change, 0 to destroy**. In a shared account, "0 to change, 0 to destroy" is your proof that you will not touch anyone else's resources.
4. `terraform apply tfplan` executes exactly that plan. Recorded: **72 resources in 12 minutes 30 seconds** (one module feature was switched off after the first plan, see chapter 4).
5. `terraform destroy` deletes them again (chapter 13).

State

The state file contains resource IDs and the account number, so it is **never committed** (`.gitignore` lists `*.tfstate`). For one operator and a short-lived lab, local state is fine. A team keeps state in **S3 with locking**, so two people cannot apply at the same time.

Where it is configured

File	Contents
<code>versions.tf</code>	Terraform $\geq$ 1.10; providers aws $\sim$ > 6.67, helm $\sim$ > 3.3, kubernetes $\sim$ > 3.3
<code>variables.tf</code>	region (validated), allowed account, repository, OIDC subject prefix, versions
<code>main.tf</code>	tags, provider configuration, token for Kubernetes/Helm
<code>vpc.tf</code>	the network (chapter 2)
<code>eks.tf</code>	the cluster and the app namespace (chapter 4)
<code>load-balancer-controller.tf</code>	IAM, Pod Identity and the Helm release (chapter 10)
<code>ecr-and-github.tf</code>	the registry and the pipeline's role (chapters 8, 9)
<code>outputs.tf</code>	values to read after apply; the ones containing the account ID are <code>sensitive</code>

Guard rails for a shared account

```
provider "aws" {  
  region          = var.region          # only eu-central-1 passes validation  
  allowed_account_ids = [var.allowed_account_id] # any other account: Terraform refuses to run  
  default_tags { tags = local.tags }  
}
```

`allowed_account_id` has no default: you set it with `export TF_VAR_allowed_account_id=<12 digits>`, so it never lands in git.

Talking to Kubernetes from Terraform

The `kubernetes` and `helm` providers authenticate to the new cluster with a short-lived token from `aws eks get-token`, using the same AWS credentials. That is how Terraform creates the app namespace and installs the load balancer controller in the same run.

Try it

```
cd terraform  
terraform init -backend=false && terraform validate # no AWS account needed  
terraform fmt -check -recursive  
export TF_VAR_allowed_account_id=<your account id>  
terraform plan -out=tfplan # read the last line before applying
```

Common mistakes

- Running `apply` without reading the plan.
- Committing `terraform.tfstate` or `*.tfvars` with real values.
- Unpinned module versions: a new module release changes your infrastructure without a code change.
- Resources created outside Terraform (here: the load balancer, created by a controller) block `destroy` later.

Check yourself

1. What does "0 to change, 0 to destroy" in a plan tell you?
2. Why is the account ID passed as an environment variable and not written in a file?
3. Why would two people running `apply` with local state be dangerous?

Answers

▼ Answers

1. That Terraform will only create new resources and will not modify or delete anything that already exists in its state.
2. To keep it out of git (and out of screenshots), and so that the same code can be used by anyone with their own account.
3. Each has their own state file: they do not see each other's changes and can create duplicates or delete each other's resources. Remote state with locking prevents this.

4. Amazon EKS

What is it?

Kubernetes runs containers on a group of machines and keeps them running. **Amazon EKS** (Elastic Kubernetes Service) is Kubernetes where AWS runs the "brain" for you.

Part	Who runs it	Job
Control plane	AWS	the Kubernetes API server, scheduler, etcd database
Managed node group	AWS creates and updates the EC2 machines; you choose size and count	the worker nodes that run your pods
Add-ons	AWS-provided, installed by you	cluster plumbing: networking, DNS, identity

Why this project uses it

Running your own Kubernetes control plane means patching, backups and high availability. EKS does that, integrates with IAM and load balancers, and is very common in job descriptions.

Alternatives: self-managed Kubernetes on EC2, Amazon ECS (simpler, AWS-only), AWS Fargate (no nodes), GKE/AKS on other clouds.

How it works: the settings that matter

Setting	Value here	Why
Kubernetes version	1.37	the newest version in standard support at build time
Subnets	private only	nodes never get public IPs
Control plane logs	api, audit, authenticator (7 days)	who did what in the cluster
Secrets encryption	a dedicated KMS key	Kubernetes secrets are encrypted with your key, not just at disk level
Authentication mode	API (access entries)	IAM identities mapped to Kubernetes permissions as code
Node group	2 × t3.medium, Amazon Linux 2023, 20 GB gp3, encrypted	small, cheap, two zones
Instance metadata	IMDSv2 required, hop limit 1	pods cannot reach the node's AWS credentials

Access entries

Old EKS clusters mapped IAM users to Kubernetes users in a ConfigMap called `aws-auth`, edited by hand.

Access entries do it through the EKS API, as code: - the IAM user that ran Terraform becomes cluster admin (`enable_cluster_creator_admin_permissions = true`), - the pipeline's role gets `AmazonEKSEditPolicy` **only in the namespace** `profile-card` (chapter 8).

IMDSv2 and the hop limit

Every EC2 machine has an **instance metadata service (IMDS)** at `169.254.169.254`. It hands out, among other things, the machine's temporary AWS credentials. If a pod can reach it, a compromised app can steal the node's credentials. - **IMDSv2 required** means you first need a session token (a `PUT` request), so simple tricks like SSRF with a `GET` do not work. - **Hop limit 1** means the token's response packet can only travel one network hop: it reaches the node itself, but not a pod behind it.

Recorded from a pod in the cluster: the token request **timed out**, and the old token-less request was answered with **401**.

Add-ons

Add-on	Job
<code>vpc-cni</code>	gives pods VPC IP addresses; here also enforces NetworkPolicies (<code>enableNetworkPolicy = "true"</code>)
<code>coredns</code>	DNS inside the cluster
<code>kube-proxy</code>	Service networking on each node
<code>eks-pod-identity-agent</code>	lets pods get AWS permissions without keys (Pod Identity , chapter 10)

Pod Identity replaces the older **IRSA** mechanism. IRSA needs an extra IAM OIDC provider per cluster; the project switches it off (`enable_irsas = false`), which is why the apply created 72 resources and not the 73 from the first plan. Fewer global IAM objects in a shared account is better.

Where it is configured

`terraform/eks.tf` (module `terraform-aws-modules/eks/aws` 21.26.0), shortened:

```

module "eks" {
  name           = local.name
  kubernetes_version = var.kubernetes_version      # "1.37"
  subnet_ids     = module.vpc.private_subnets

  endpoint_public_access = true          # see chapter 6: an accepted risk
  endpoint_private_access = true

  enabled_log_types      = ["api", "audit", "authenticator"]
  encryption_config      = { resources = ["secrets"] }
  kms_key_deletion_window_in_days = 7

  enable_irs          = false
  authentication_mode  = "API"
  enable_cluster_creator_admin_permissions = true

  eks_managed_node_groups = {
    default = {
      instance_types = [var.node_instance_type] # t3.medium
      min_size       = 2
      max_size       = 3
      desired_size    = 2
      metadata_options = { http_tokens = "required", http_put_response_hop_limit = 1 }
    }
  }
}

```

The same file creates the app **namespace** with Pod Security `restricted` (chapter 11) and the readiness gate label (chapter 12).

Recorded timings: control plane **8 min 44 s**, node group **1 min 59 s**. `kubectl get nodes -o wide` showed two Ready nodes, v1.37.0, in 10.42.14.x and 10.42.30.x (two zones), with **EXTERNAL-IP** `<none>`.

Try it

```

aws eks update-kubeconfig --name eks-devsecops --region eu-central-1
kubectl get nodes -o wide
kubectl get pods -n kube-system
aws eks describe-cluster --name eks-devsecops \
  --query '{logs: cluster.logging.clusterLogging[?enabled].types | [0], auth:
cluster.accessConfig.authenticationMode}'

```

Use a `kubectl` within one minor version of the cluster (1.36–1.38 for 1.37).

Common mistakes

- Leaving IMDS reachable from pods (hop limit 2 is the default for many setups).
- Editing `aws-auth` by hand and locking yourself out.
- Giving the CI pipeline `cluster-admin` "to make it work".
- A `kubectl` that is too old or too new for the cluster version.

Check yourself

1. What does AWS run for you in EKS, and what do you still decide?
2. What do access entries replace?
3. Why does a hop limit of 1 protect the node's credentials from pods?

Answers

▼ Answers

1. AWS runs the control plane (API server, etcd, scheduler) and manages the node machines' lifecycle; you decide the version, networking, node size and count, add-ons and permissions.
2. The `aws-auth` ConfigMap: IAM-to-Kubernetes mappings are now made through the EKS API (as code), with scoped access policies.
3. The IMDSv2 session token's response can only travel one network hop. A pod's traffic passes an extra hop (its own network namespace), so it never receives the token and cannot read the credentials.

5. Containers for the app

What is it?

- **React** is a JavaScript library for user interfaces. **Vite** builds a React app into plain static files: one `index.html`, plus JavaScript and CSS files with a hash in their names (`index-xTPJYkoq.js`).
- A **container image** packages an app with everything it needs to run. A **Dockerfile** is the recipe.
- **nginx** is a fast web server; **nginx-unprivileged** is the official variant that runs as a normal user on port 8080.

Why this project builds the image this way

The app is just static files. It needs no Node.js at runtime, no shell, no root. Every extra component is something to patch and something an attacker can use. So the image is: - built in **two stages** (build with Node.js, run with nginx only), - based on images **pinned by digest**, - running as a **non-root** user, with a **read-only** file system in Kubernetes, - sending **strict security headers** with every response.

How it works

The app and its tests

`app/src/App.jsx` renders the profile card from `app/src/profile.js`. The footer shows the version and git commit, which the pipeline bakes into each build (`VITE_VERSION`, `VITE_COMMIT`), so you always see what is running.

`app/src/App.test.jsx` has **5 tests** (Vitest + Testing Library): name and role, every skill listed, external links open safely in a new tab, mailto links do not, the build footer is shown. CI also runs `oxlint` and `npm audit --audit-level=high`.

The multi-stage Dockerfile

`app/Dockerfile`:

```

FROM node:24-alpine@sha256:ebfe2f90...05ec1c1 AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --no-audit --no-fund
COPY index.html vite.config.js ./
COPY public ./public
COPY src ./src
ARG VERSION=dev
ARG COMMIT=local
ENV VITE_VERSION=${VERSION} VITE_COMMIT=${COMMIT}
RUN npm run build

FROM nginxinc/nginx-unprivileged:1.30-alpine@sha256:ed04ec1f...dd20b1e
USER 0
RUN apk upgrade --no-cache
USER 101
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
COPY nginx/security-headers.conf /etc/nginx/snippets/security-headers.conf
COPY --from=build /app/dist /usr/share/nginx/html
USER 101
EXPOSE 8080

```

- **Stage 1** installs dependencies with `npm ci` (exactly the versions in `package-lock.json`) and builds the static files.
- **Stage 2** starts from nginx and copies only the built files. Node.js and all build tools stay behind.
- `USER 0` → `apk upgrade` → `USER 101`: apply Alpine security fixes that are newer than the base image, then drop back to the unprivileged user. Chapter 6 shows the real vulnerability that made this necessary.
- **Digests** (`@sha256:...`) pin the exact base image content. A tag like `1.30-alpine` can be moved to different content; a digest cannot.

nginx configuration

`app/nginx/default.conf`: - `listen 8080` and `server_tokens off` (no version in error pages) - `location = /healthz` returns `ok`: used by Kubernetes probes and by the load balancer health check - `/assets/` (hashed files) are cached for a year; everything else returns `index.html` (single-page app routing) with `Cache-Control: no-cache`

`app/nginx/security-headers.conf` is included in every location, because nginx does **not** inherit `add_header` into a location that sets its own headers:

Header	Effect
<code>Content-Security-Policy: default-src 'self'; ...</code>	the browser only runs scripts and styles from this site
<code>X-Content-Type-Options: nosniff</code>	no guessing of file types
<code>X-Frame-Options: DENY, frame-ancestors 'none'</code>	the page cannot be embedded (clickjacking)
<code>Referrer-Policy: no-referrer</code>	no URL leaks to other sites
<code>Permissions-Policy: camera=(), microphone=(), ...</code>	no access to device features

These headers came back from the real load balancer: `curl -D -` showed all of them with `HTTP/1.1 200 OK`.

Try it (locally, no AWS)

```
docker build -t profile-card:local --build-arg VERSION=0.1.0 --build-arg COMMIT=abcdef1 app
docker run -d --name pc --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges \
  -p 18080:8080 profile-card:local
curl -s localhost:18080/healthz
curl -sI localhost:18080/ | grep -i -E 'content-security|x-frame'
docker exec pc id          # uid=101(nginx)
docker rm -f pc
```

Open <http://localhost:18080> in a browser: the footer shows `v0.1.0 · abcdef1`.

Common mistakes

- Single-stage images that ship Node.js, npm and the source code to production.
- `npm install` instead of `npm ci` in builds (versions drift).
- Adding `add_header` inside one nginx location and silently losing all the security headers there.
- Running the web server as root "because port 80 needs it". Use port 8080 and let the Service map port 80.

Check yourself

1. What does stage 2 of the Dockerfile copy from stage 1?
2. Why is the image pinned by digest even though the tag `1.30-alpine` exists?
3. Why does the security header snippet have to be included in every nginx location?

Answers

▼ Answers

1. Only the built static files (`/app/dist`), nothing else.
2. A tag can be moved to different content at any time; the digest is the hash of the content and guarantees exactly the image that was tested and scanned.
3. nginx does not inherit `add_header` directives into a location that defines its own `add_header` (here: `Cache-Control`), so the headers would disappear from those responses.

6. Trivy and scanning

What is it?

Trivy (by Aqua Security, open source) is a security scanner. In this project it does two jobs:

Mode	Command	Finds
Image scan	<code>trivy image</code>	known vulnerabilities (CVEs) in the operating-system packages and libraries inside an image
Misconfiguration scan	<code>trivy config</code>	risky settings in Terraform and Kubernetes files (public endpoints, root containers, missing limits...)

A **CVE** (Common Vulnerabilities and Exposures) is the public ID of a known security bug, for example `CVE-2026-103111`. Each has a **severity**: LOW, MEDIUM, HIGH, CRITICAL.

Why this project uses it

A vulnerability that already has a fix should never reach production. And a misconfiguration in Terraform is much cheaper to fix before `apply` than after an incident.

Alternatives: Gype (images), Snyk, Docker Scout, Amazon Inspector (paid ECR scanning); Checkov, tfsec (now part of Trivy) and kube-score for configuration.

How it works

The image gate

```
trivy image --quiet --exit-code 1 --severity CRITICAL,HIGH --ignore-unfixed profile-card:build
```

- `--severity CRITICAL,HIGH` : only these block the build.
- `--ignore-unfixed` : block only on problems that **have a fix**. A gate that fails on things nobody can fix just teaches people to switch it off.
- `--exit-code 1` : any finding makes the command fail, which fails the pipeline step.

The real finding

The very first image of this project was **blocked**. The official `nginx-unprivileged:1.30-alpine` base image, pinned to the latest digest, contained:

```
profile-card:local (alpine 3.24.2)
Total: 1 (HIGH: 1, CRITICAL: 0)
| pcre2 | CVE-2026-103111 | HIGH | fixed | 10.48-r0 | 10.49-r0 | Out-of-bounds write via crafted regular
expression
gate exit code: 1
```

The fix already existed in Alpine; the base image was just older than the fix. Three lines in the Dockerfile (`USER 0`, `RUN apk upgrade --no-cache`, `USER 101`) and the scan showed **0 vulnerabilities**, exit code 0.

Misconfiguration scans

CI scans two things: 1. the Terraform code (skipping `terraform/.terraform`, where the downloaded modules' own examples live), 2. the **rendered** Kubernetes manifests (`kubect1 kustomize k8s`), so Trivy sees what is really applied.

Results in this project: - Kubernetes: 99 checks passed. Two LOW findings ("runs with $\text{UID} \leq 10000$ ") were fixed by running as UID **10101**. - **KSV-0125 (untrusted registry)** caught a test placeholder `registry.example` in the CI render. The real deployment uses ECR, so the test now uses an ECR-style address. - Terraform: **three real findings in this project's configuration**, which were kept on purpose.

Written-down exceptions

`.trivyignore.yaml`:

ID	Finding	Why accepted	Production fix
AWS-0040	EKS API endpoint is public	GitHub-hosted runners have no fixed IP; access still needs IAM	private endpoint + self-hosted runners in the VPC
AWS-0041	public endpoint open to 0.0.0.0/0	same reason	same
AWS-0104	nodes may connect out to any IP	nodes need ECR, EKS, STS and package mirrors via NAT; pod egress is denied separately	egress through a proxy or VPC endpoints

Each entry has a **statement** (the reason) and an `expired_at: 2027-04-01`. On that date the pipeline fails again and someone must review it. An exception without a reason and an end date quietly becomes permanent.

A second opinion in AWS

ECR also scans every pushed image (**scan on push**, basic scanning, free). The deploy job waits for that result and stops on any CRITICAL finding (chapter 9).

Where it is integrated

- `.github/workflows/ci.yaml`: job `container` (image gate), job `infrastructure` (Terraform and Kubernetes scans).
- `.github/workflows/deploy.yaml`: job `build-scan` (image gate before anything is pushed).
- Trivy itself is downloaded by `scripts/ci/install-tools.sh` and checked against a pinned SHA-256.

Try it (locally)

```
docker build -t profile-card:local app
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.75.0 \
  image --severity CRITICAL,HIGH --ignore-unfixed profile-card:local
docker run --rm -v "$PWD:/src" -w /src aquasec/trivy:0.75.0 \
  config --severity HIGH,CRITICAL --ignorefile .trivyignore.yaml --skip-dirs terraform/.terraform
terraform
```

On Windows (Git Bash) use `//var/run/docker.sock` and `MSYS_NO_PATHCONV=1`.

Common mistakes

- Blocking on unfixed vulnerabilities: the gate is always red and people stop looking.
- Scanning Terraform including `.terraform/` : you get findings from module examples you do not use.
- Ignoring findings with no reason and no end date.
- Scanning the image *after* pushing it: then a vulnerable image is already in the registry.

Check yourself

1. What does `--ignore-unfixed` change, and why is it useful?
2. Why was the first image blocked, and how was it fixed?
3. What two things must every accepted exception have?

Answers

▼ Answers

1. Only vulnerabilities that already have a fixed version are reported, so the gate fails only on problems the team can actually fix.
2. The nginx base image contained pcre2 10.48 with CVE-2026-103111 (HIGH, fixed in 10.49). Running `apk upgrade` in the runtime stage installed the fixed package.
3. A written reason (with the real fix) and an expiry date, so it is reviewed again.

7. GitHub Actions CI

What is it?

GitHub Actions runs **workflows** (YAML files in `.github/workflows/`) on GitHub's machines when something happens in the repository: a push, a pull request, a button click. A workflow has **jobs**; a job has **steps**; a step runs a command or an **action** (a reusable step published by someone).

CI (continuous integration) means: every change is checked automatically before it is merged.

Why this project uses it

The code lives on GitHub, Actions is free for public repositories, and it can get short-lived cloud credentials through OIDC (chapter 8).

Alternatives: GitLab CI, Jenkins, CircleCI, Azure Pipelines, AWS CodePipeline.

How it works: the five CI jobs

`.github/workflows/ci.yaml` runs on every pull request and every push to `main`. **None of its jobs can reach AWS.**

Job	Steps	Catches
Secret scan	<code>gitleaks git --redact --exit-code 1 .</code> on the full history	access keys, passwords and tokens committed by mistake, even in old commits
App	<code>npm ci</code> , <code>npm run lint</code> (oxlint), <code>npm test</code> (5 tests), <code>npm audit --audit-level=high</code> , <code>npm run build</code>	broken code, failing tests, vulnerable npm packages
Container	hadolint on the Dockerfile, <code>docker build</code> , Trivy image gate	Dockerfile mistakes, fixable HIGH/CRITICAL CVEs
Infrastructure	<code>terraform fmt -check</code> , <code>init -backend=false</code> , <code>validate</code> , Trivy config (Terraform), render manifests, <code>kubeconform -strict</code> , Trivy config (Kubernetes)	invalid Terraform, risky settings, manifests that do not match the Kubernetes schema
Pipeline security	<code>check-pinned-actions.sh</code> , actionlint, zizmor	unpinned actions, workflow syntax errors, workflow security mistakes

Securing the pipeline itself

The pipeline holds the keys to production, so it is protected like production:

- `permissions: {}` at the top: no job gets any token permission unless it asks. Each CI job asks only for `contents: read`.
- Actions pinned to a full commit SHA**, for example `actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1`. A version tag like `v7` can be moved to a malicious commit (this happened to the popular `tj-actions/changed-files` action in 2025); a commit SHA cannot. `scripts/ci/check-pinned-actions.sh` fails CI if anyone adds an unpinned action.
- `persist-credentials: false` on checkout: the job's token is not left on disk for later steps.

- **Tools installed by checksum** ([scripts/ci/install-tools.sh](#)): gitleaks, Trivy, Syft, cosign, kubeconform, Terraform and actionlint are downloaded from their official releases and checked against pinned SHA-256 values, instead of adding more third-party actions.
- **actionlint** checks workflow syntax and the shell scripts inside `run:` (it found a word-splitting issue during this project).
- **zizmor** audits workflows for security problems such as template injection (`${{ }}` inside `run:`), excessive permissions and credential persistence. It reported **no findings** for both workflows.
- **CODEOWNERS** and **Dependabot** ([.github/](#)): changes to workflows, Terraform and manifests need the owner's review; Dependabot proposes updates for actions, npm, Docker base images and Terraform.

Where it is configured

```
name: ci
on:
  pull_request:
  push:
    branches: [main]
permissions: {}

jobs:
  secrets:
    runs-on: ubuntu-24.04
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          fetch-depth: 0          # gitleaks needs the full history
          persist-credentials: false
      - run: scripts/ci/install-tools.sh gitleaks
      - run: gitleaks git --redact --no-banner --exit-code 1 .
```

Try it

```
scripts/ci/check-pinned-actions.sh
docker run --rm -v "$PWD:/repo" -w /repo rhysd/actionlint:1.7.12 -color=false
docker run --rm -v "$PWD:/repo" -w /repo python:3.13-slim sh -c \
  "pip install -q --require-hashes -r tools/zizmor-requirements.txt && zizmor --min-severity medium
  .github/workflows"
docker run --rm -i hadolint/hadolint:v2.15.1 < app/Dockerfile
```

Then make a mistake on purpose (lab 3) and watch which check catches it.

Common mistakes

- `permissions: write-all` or no `permissions` block at all.
- Using actions by tag (`@v4`) instead of commit SHA.
- `${{ github.event.pull_request.title }}` directly inside `run:` (template injection).
- Scanning only the latest commit for secrets: a key deleted in the next commit is still in the history.

Check yourself

1. Why does the secret scan check out the full history?
2. What is the difference between a version tag and a commit SHA for an action?
3. Which CI job would fail if someone added `USER root` at the end of the Dockerfile?

Answers

▼ Answers

1. A secret committed once stays in git history even if a later commit removes it; scanning only the latest files would miss it.
2. A tag is a movable label the action's owner (or an attacker with their access) can point at any commit; a full commit SHA always means exactly the same code.
3. The **container** job: hadolint rule DL3002 ("last USER should not be root") fails it. In the cluster the pod would still run as UID 10101, because `runAsUser` in the Deployment overrides the image's `USER` (chapter 11): two independent layers. Lab 3 lets you try it.

8. OIDC and least privilege

What is it?

- **OIDC** (OpenID Connect) is a standard for proving identity with signed **tokens**. GitHub Actions can give each job a token that says, in a signed way: "I am a job of repository X, running workflow Y, on branch/environment Z".
- AWS can **trust** GitHub as an **identity provider**: a job sends its token to AWS STS (`AssumeRoleWithWebIdentity`) and receives **temporary credentials** for an IAM role, valid for one hour at most.
- **Least privilege** means every identity gets only the permissions it needs, nothing more.

Why this project uses it

The classic way is to create an IAM user, generate an access key and store it as a GitHub secret. Those keys never expire on their own, get copied around, and leaked cloud keys are one of the most common causes of cloud breaches. With OIDC **there is no AWS key to steal**: the credentials are created per job and expire.

How it works

1. The deploy job has `permissions: id-token: write`, so it may request a token.
2. `aws-actions/configure-aws-credentials` (pinned to a commit SHA) sends the token to AWS STS with the role ARN.
3. AWS checks the token's signature against the **identity provider** `token.actions.githubusercontent.com`, then checks the role's **trust policy** conditions.
4. If everything matches, STS returns credentials valid for this job only.

The trust policy

`terraform/ecr-and-github.tf`:

```
# The provider is shared by the whole account: read it (default), or create it in a fresh account.
data "aws_iam_openid_connect_provider" "github" {
  count = var.create_github_oidc_provider ? 0 : 1
  url   = "https://token.actions.githubusercontent.com" # existing and shared: read, never changed
}

data "aws_iam_policy_document" "github_trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type       = "Federated"
      identifiers = [local.github_oidc_provider_arn] # the read or the created provider
    }
    condition {
      test      = "StringEquals"
      variable  = "token.actions.githubusercontent.com:aud"
      values    = ["sts.amazonaws.com"]
    }
    condition {
      test      = "StringEquals"
      variable  = "token.actions.githubusercontent.com:sub"
      values    = ["${var.github_oidc_subject_prefix}:environment:production"]
    }
  }
}
}
```

- **aud (audience)** must be `sts.amazonaws.com`: the token was meant for AWS.
- **sub (subject)** must be this repository, running in its **production environment**. That GitHub environment only accepts deployments from the `main` branch. A pull request, a fork or another branch gets a different subject and is refused.
- The account already had a GitHub identity provider used by other teams, so Terraform only **reads** it with a data source. It is never modified or deleted.

The real failure: GitHub's immutable subject

The first deployment failed:

```
Could not assume role with OIDC: Not authorized to perform sts:AssumeRoleWithWebIdentity
```

The trust policy expected `repo:sufyanahmadkamboh/sufyan-devops-eks-devsecops:environment:production`. But new GitHub repositories use an **immutable subject** that includes the numeric IDs of the owner and the repository:

```
repo:sufyanahmadkamboh@25397028/sufyan-devops-eks-devsecops@1403679100:environment:production
```

You can read it with `gh api repos/OWNER/REPO/actions/oidc/customization/sub`. This is a security improvement: if someone deletes the repository and creates a new one with the same name, it gets a different ID and **cannot** assume the role. The fix was one variable, `github_oidc_subject_prefix`.

What the role may do in AWS

Simplified from `data "aws_iam_policy_document" "github_deploy"`:

```
statement { actions = ["ecr:GetAuthorizationToken"] resources = ["*"] } # no resource-level support
statement {
  actions = ["ecr:BatchCheckLayerAvailability", "ecr:PutImage", "ecr:InitiateLayerUpload",
    "ecr:UploadLayerPart", "ecr:CompleteLayerUpload", "ecr:BatchGetImage",
    "ecr:DescribeImages", "ecr:DescribeImageScanFindings", "ecr:GetDownloadUrlForLayer"]
  resources = [aws_ecr_repository.app.arn]
}
statement { actions = ["eks:DescribeCluster"] resources = [module.eks.cluster_arn] }
```

What the role may do in Kubernetes

An EKS **access entry** ([terraform/eks.tf](#)) maps the role to `AmazonEKSEditPolicy` with `access_scope = { type = "namespace", namespaces = ["profile-card"] }`: it may create and change Deployments, Services, Ingresses and so on, **only in** `profile-card`. It cannot even create that namespace; Terraform (the platform) owns it.

Proving it on every deploy

The deploy job checks its own permissions with `kubectl auth can-i` and fails if any answer is unexpected. Recorded output:

```
can-i create deployments -n profile-card -> yes (expected yes)
can-i create deployments -n kube-system -> no (expected no)
can-i get secrets -n kube-system -> no (expected no)
can-i create clusterrolebindings -> no (expected no)
can-i delete namespaces -> no (expected no)
```

Try it

```
gh api repos/<you>/<your-fork>/actions/oidc/customization/sub # your subject prefix
terraform -chdir=terraform output -raw github_deploy_role_arn | gh secret set AWS_DEPLOY_ROLE_ARN --env
production
aws iam get-role --role-name eks-devsecops-github-deploy --query 'Role.AssumeRolePolicyDocument'
```

Common mistakes

- A trust policy with `StringLike` and `repo:owner/*`: every repository of the owner (and every branch) can assume the role.
- Forgetting the `aud` condition.
- Using the old `repo:owner/name` subject for a new repository (the failure above).
- Giving the deploy role `AdministratorAccess` and the cluster-admin policy "for now".
- Deleting or "fixing" a shared identity provider in a company account.

Check yourself

1. What is stored in GitHub so the pipeline can reach AWS?
2. Why does the subject include `environment:production`?
3. Name two things the deploy role cannot do in the cluster, and how that is proven.

Answers

▼ Answers

1. Only the role's ARN (as a secret, to keep the account ID out of public logs). No access key, no password.
2. Only jobs that run in the protected `production` environment, which accepts only the `main` branch, get that subject. Pull requests and other branches cannot assume the role.
3. For example: create deployments in `kube-system`, read secrets in `kube-system`, create cluster role bindings, delete namespaces. Every deploy runs `kubectl auth can-i` for these and fails if any answer is "yes".

9. The deploy pipeline

What is it?

CD (continuous deployment): every change that reaches `main` and passes all checks is deployed automatically. Here that is `.github/workflows/deploy.yaml`. It runs on every push to `main` that touches `app/`, `k8s/` or the workflow, and can be started by hand.

Why it is built this way

Two jobs, because the riskiest moment is when tools and credentials meet:

Job	Has AWS credentials?	Does
1. Build and scan	no	build the image (with version and commit), Trivy gate, SBOM, save the image as an artifact
2. Deploy to EKS	yes, temporary (OIDC), in the production environment	push, ECR scan, sign, deploy, check permissions, smoke test

- The scanners never run next to AWS credentials.
- A vulnerable image never even reaches the registry.
- Job 2 deploys exactly the image job 1 scanned (it is passed as an artifact, `image.tar.gz`).

How it works: job 2 step by step

1. **OIDC → AWS** (chapter 8). The account ID is masked in the public logs (`mask-aws-account-id: true` and `::add-mask::`).
2. **Push to ECR by digest**. The image gets an immutable tag like `1.0.10-cddef1d` ; the step outputs the **digest** (`sha256:...`) that the registry computed. Everything after this uses the digest.
3. **ECR scan on push (second opinion)**. ECR scans the image; the job stops on any CRITICAL finding.
4. **cosign keyless signature**. `cosign sign` signs the digest with a short-lived certificate for this workflow (Sigstore), then `cosign verify` checks it with the expected identity `.../.github/workflows/deploy.yaml@refs/heads/main` . The signature is stored next to the image in ECR (the untagged entries in the image list).
5. **Deploy the exact digest**. `k8s/kustomization.yaml` contains placeholders; the job replaces them and applies: `yaml images:`
 - `name: profile-card newName: IMAGE_REPOSITORY # → /eks-devsecops/profile-card digest: IMAGE_DIGEST # → sha256:...` `bash sed -i -e "s|IMAGE_REPOSITORY|${REPO}|" -e "s|IMAGE_DIGEST|${DIGEST}|" k8s/kustomization.yaml kubectl apply -k k8s kubectl -n profile-card rollout status deployment/profile-card --timeout=300s `` Pods run .../profile-card@sha256:...`, never a tag.`
6. **Least privilege check** with `kubectl auth can-i` (chapter 8).
7. **Smoke test through the load balancer**. The job reads the Ingress hostname and waits until `/healthz` answers and the page contains "Profile". GitHub shows the URL on the run page.

Recorded run triggered by a push: **3 minutes 37 seconds** from push to live (build and scan 33 s, deploy 2 min 54 s).

Real bugs found in the pipeline

Pipelines are code, and they need testing too. Two real problems appeared in step 3:

1. `aws ecr wait image-scan-complete` **failed with** `ScanNotFoundException`. Right after a push, ECR has not registered the scan yet, and the CLI waiter gives up instead of waiting. Fix: a loop that polls `describe-image-scan-findings` until the status is `COMPLETE` (or `FAILED` / `UNSUPPORTED_IMAGE`).
2. **A clean image failed the check.** With zero findings, ECR returns no `findingSeverityCounts` at all, so the CLI printed nothing and the `jq` check treated "nothing" as an error. Fix: treat an empty answer as `{}`, and read `(. // {}).CRITICAL // 0`.

```
counts="$(aws ecr describe-image-scan-findings ... --query 'imageScanFindings.findingSeverityCounts' --output json)"
counts="${counts:-null}"
[[ "$(jq -r '(. // {}).CRITICAL // 0' <<<"${counts}")" == 0 ]]
```

Switching deployments on and off

Job 2 only runs when the repository variable `DEPLOY_ENABLED` is `true`. It is switched on while the lab cluster exists and off before the teardown, so a push can never try to deploy to a deleted cluster.

Common mistakes

- Scanning after pushing, or deploying a tag that was scanned yesterday.
- `kubectl set image ...:latest`: you no longer know what runs.
- No rollout wait: the pipeline is green while the new pods crash.
- No smoke test through the real entry point (the load balancer).
- Trusting AWS CLI waiters blindly: read what they do on errors.

Check yourself

1. Why is the image passed from job 1 to job 2 as an artifact instead of being rebuilt?
2. What does the cluster run: a tag or a digest? Where does the digest come from?
3. What was wrong with the first version of the ECR scan check?

Answers

▼ Answers

1. So that job 2 deploys exactly the bytes job 1 scanned; a rebuild could produce a different image (new base image packages, new dependencies).
2. A digest. The registry computes it during the push; the job reads it with `docker inspect` (`RepoDigests`) and writes it into the kustomization.
3. Two things: the waiter failed when the scan was not registered yet (`ScanNotFoundException`), and a clean image (no findings, so no counts) was treated as a failure.

10. Load Balancer Controller and Ingress

What is it?

- A Kubernetes **Service** gives a group of pods one stable address inside the cluster.
- An **Ingress** describes how HTTP traffic from outside reaches Services ("everything under `/` goes to the Service `profile-card`").
- An Ingress does nothing by itself: an **ingress controller** reads it and makes it real. The **AWS Load Balancer Controller** makes it real as an **Application Load Balancer (ALB)**, a managed HTTP load balancer from AWS.
- **EKS Pod Identity** gives a pod (here the controller) AWS permissions through an IAM role, without any access keys in the cluster.

Why this project uses it

The ALB is managed, highly available across zones, integrates with AWS certificates and WAF, and the controller keeps it in sync with Kubernetes automatically.

Alternatives: a Service of type `LoadBalancer` (a Network Load Balancer per Service), ingress-nginx behind one load balancer, Gateway API implementations.

How it works

1. Terraform installs the controller with **Helm** (chart 3.5.0, 2 replicas, in `kube-system`).
2. The controller's service account `aws-load-balancer-controller` is linked to the IAM role `eks-devsecops-lb-controller` by a **Pod Identity association**. The role's permissions come from the controller's official policy file for version 3.5.0.
3. The pipeline applies the Ingress. The controller sees `ingressClassName: alb` and creates an ALB, a target group, listeners and security groups.
4. With **target type** `ip`, the ALB sends traffic straight to the **pod IP addresses** (possible because pods have VPC IPs), not through node ports.
5. The ALB is placed in subnets tagged `kubernetes.io/role/elb` (the public subnets, chapter 2).

Why the nodes block instance metadata, and what that means for the controller

The controller would normally ask the instance metadata service for the VPC ID and region. With the hop limit of 1 (chapter 4) pods cannot reach it, so Terraform passes `vpcId` and `region` to the Helm chart directly.

Where it is configured

`terraform/load-balancer-controller.tf`:

```

resource "aws_eks_pod_identity_association" "lb_controller" {
  cluster_name = module.eks.cluster_name
  namespace    = "kube-system"
  service_account = "aws-load-balancer-controller"
  role_arn      = aws_iam_role.lb_controller.arn
}

resource "helm_release" "lb_controller" {
  chart = "aws-load-balancer-controller"
  version = "3.5.0"
  values = [yamlencode({
    clusterName = module.eks.cluster_name
    region      = var.region
    vpcId       = module.vpc.vpc_id
    replicaCount = 2
    defaultTags = local.tags # the ALB and its security groups get the project tags too
  })]
}

```

k8s/ingress.yaml:

```

metadata:
  annotations:
    alb.ingress.kubernetes.io/load-balancer-name: eks-devsecops-profile-card
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
    alb.ingress.kubernetes.io/healthcheck-path: /healthz
    alb.ingress.kubernetes.io/target-group-attributes: deregistration_delay.timeout_seconds=10
    alb.ingress.kubernetes.io/load-balancer-attributes: >-
      routing.http.drop_invalid_header_fields.enabled=true,routing.http.desync_mitigation_mode=strictest
spec:
  ingressClassName: alb

```

- `drop_invalid_header_fields` and `desync_mitigation_mode=strictest` protect against HTTP request smuggling at the edge.
- The deregistration delay is explained in chapter 12.

Recorded result: the Ingress got the address `eks-devsecops-profile-card-604868946.eu-central-1.elb.amazonaws.com`, and `curl` returned `HTTP/1.1 200 OK` with all security headers.

Why HTTP only

HTTPS needs a TLS **certificate**, and a certificate is issued for a **domain name**. This lab has no domain, so it runs on plain HTTP. With a domain you request a certificate from **AWS Certificate Manager (ACM)** and add two annotations (`certificate-arn` and an HTTPS listener with a redirect from HTTP).

Teardown note

The ALB is created by the **controller**, not by Terraform. If you run `terraform destroy` first, the controller is deleted before it can remove the ALB, and the ALB plus its security groups block the deletion of the VPC. So the teardown deletes the Ingress first (chapter 13).

Try it

```
kubectl -n kube-system get deploy aws-load-balancer-controller
kubectl -n profile-card get ingress profile-card
H=$(kubectl -n profile-card get ingress profile-card -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
curl -sS -D - -o /dev/null "http://$H/"
aws elbv2 describe-load-balancers --names eks-devsecops-profile-card --query 'LoadBalancers[0].Scheme'
```

Common mistakes

- Missing subnet tags: the Ingress never gets an address.
- Giving the controller credentials as Kubernetes secrets instead of Pod Identity.
- Forgetting `vpcId / region` when IMDS is blocked: the controller cannot start.
- Running `terraform destroy` before deleting the Ingress.

Check yourself

1. What creates the ALB, Terraform or the controller?
2. What does `target-type: ip` change?
3. What is missing for HTTPS, and how would you add it?

Answers

▼ Answers

1. The AWS Load Balancer Controller, when it sees the Ingress. Terraform only installs the controller.
2. The ALB registers the pod IP addresses directly as targets, instead of sending traffic to node ports and through kube-proxy.
3. A domain name and a certificate for it. Request one in ACM, add the certificate ARN and an HTTPS listener (with an HTTP→HTTPS redirect) as Ingress annotations.

11. Kubernetes workload security

What is it?

Even a perfectly scanned image can be run dangerously. Kubernetes has three tools to limit what a running pod can do:

Tool	Limits
securityContext	what the container process may do: user ID, privileges, Linux capabilities, file system
Pod Security Admission (PSA)	a built-in check that refuses pods which break a security level (<code>privileged</code> , <code>baseline</code> , <code>restricted</code>) in a namespace
NetworkPolicy	which network connections pods may receive and make

Why this project uses them

If an attacker finds a bug in the app, these settings decide how far they get: not root, no tools to download, no way out to the internet, no way to other apps.

How it works

securityContext

`k8s/deployment.yaml` :

```
automountServiceAccountToken: false
securityContext:
  runAsNonRoot: true
  runAsUser: 10101
  runAsGroup: 10101
  seccompProfile: {type: RuntimeDefault}
containers:
- name: web
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities: {drop: [ALL]}
  resources:
    requests: {cpu: 50m, memory: 32Mi}
    limits: {cpu: 200m, memory: 64Mi}
  volumeMounts:
    - {name: tmp, mountPath: /tmp}
```

Setting	Meaning
<code>automountServiceAccountToken: false</code>	no Kubernetes API token inside the pod; the app never calls the API
<code>runAsNonRoot</code> , <code>runAsUser: 10101</code>	not root; a UID above 10000 never collides with a user on the node (Trivy KSV-0020/0021)

Setting	Meaning
<code>seccompProfile: RuntimeDefault</code>	blocks dangerous Linux system calls
<code>allowPrivilegeEscalation: false</code>	no <code>sudo</code> / <code>setuid</code> tricks to gain more rights
<code>readOnlyRootFilesystem: true</code>	nothing can be written except <code>/tmp</code> (a 16 Mi <code>emptyDir</code>)
<code>capabilities: drop [ALL]</code>	no special Linux powers (raw sockets, changing file owners...)
resource limits	one pod cannot starve its neighbours

The rendered manifests passed **99 Trivy checks**.

Pod Security Admission: restricted

The namespace is created by Terraform with the label `pod-security.kubernetes.io/enforce: restricted` .
Recorded attack, as cluster admin:

```
$ kubectl -n profile-card run attacker --image=busybox:1.37 --privileged --restart=Never -- sleep 60
Error from server (Forbidden): pods "attacker" is forbidden: violates PodSecurity "restricted:latest":
privileged, allowPrivilegeEscalation != false, unrestricted capabilities, runAsNonRoot != true,
seccompProfile
```

Even an admin cannot start a privileged pod in that namespace by accident.

NetworkPolicy, and a real finding

`k8s/networkpolicy.yaml` starts with **default deny**: no traffic in, no traffic out, for every pod in the namespace.
Then one exception for HTTP from the load balancer. On EKS, policies are enforced by the **VPC CNI** add-on
(`enableNetworkPolicy = "true"` , chapter 4).

Recorded: from inside an app pod, `wget http://example.com` failed with `bad address 'example.com'` . Even DNS is blocked, so an attacker cannot download tools or send data out.

The first version had a weakness. Its ingress rule allowed port 8080 from the whole VPC (`10.42.0.0/16`),
because the load balancer lives in the VPC. From a pod in another namespace:

```
$ kubectl -n default run lateral-test --image=curlimages/curl -- curl http://<profile-card pod IP>:8080/
from another namespace: HTTP 200
```

The reason: with the VPC CNI, **pod IPs are VPC IPs**. "Allow the VPC" also meant "allow every pod in the cluster". This is called **lateral movement**: an attacker in one app moving to another.

The fix: allow only the **public subnets**, where the ALB's network interfaces live:

```
ingress:
  - from:
      - ipBlock: {cidr: 10.42.48.0/23} # public subnets, where the ALB's network interfaces live
    ports:
      - {protocol: TCP, port: 8080}
```

It was tested live before committing: the pods stayed ready for 90 seconds (kubelet health probes still work),
the ALB kept answering 200, and the lateral request **timed out**: `from another namespace: blocked` . Then the
fix went through the pipeline like any change.

Try it

```
kubectl get ns profile-card --show-labels
kubectl -n profile-card run attacker --image=busybox:1.37 --privileged --restart=Never -- sleep 60
P=$(kubectl -n profile-card get pods -o jsonpath='{.items[0].metadata.name}')
kubectl -n profile-card exec "$P" -- wget -T 5 -q -O /dev/null http://example.com
IP=$(kubectl -n profile-card get pods -o jsonpath='{.items[0].status.podIP}')
kubectl -n default run lateral --image=curlimages/curl:8.17.0 --restart=Never --rm -i -- curl -m 5
"http://$IP:8080/"
```

Common mistakes

- A NetworkPolicy without a CNI that enforces it: the YAML is accepted and does nothing.
- "Allow the VPC/cluster CIDR" rules on EKS (the finding above).
- `readOnlyRootFilesystem: true` without a writable `/tmp`: nginx cannot start.
- Labelling namespaces `warn` only: violations are reported but still allowed.

Check yourself

1. What does the default-deny policy block for an app pod, and how did the test show it?
2. Why did the "allow the VPC" rule allow a pod from another namespace?
3. Who refused the privileged pod: Kyverno, the NetworkPolicy or Pod Security?

Answers

▼ Answers

1. All incoming and outgoing traffic except the explicit exception. The app pod could not even resolve `example.com` (DNS is outgoing traffic).
2. On EKS with the VPC CNI, pods get IP addresses from the VPC's subnets, so every pod's IP is inside the VPC CIDR.
3. Pod Security Admission, because the namespace enforces the `restricted` level. (This project has no Kyverno.)

12. Zero-downtime deployments

What is it?

A **rolling update** replaces pods one by one: start a new pod, wait until it is ready, stop an old one. In theory users never notice. Behind a load balancer there is a race: the load balancer and Kubernetes do not instantly agree on which pods should get traffic.

Why it matters

Each failed request is a real user seeing an error page. During many deployments a day, small error rates add up.

How it was measured

While the pipeline deployed a new version, a small script requested the site through the ALB **twice per second** and counted the answers.

Before the fix (plain rolling update, `maxUnavailable: 0`, `maxSurge: 1`):

```
seconds: 155    200 OK: 277    502: 1    timeouts: 2    → 3 of 280 requests failed
```

What happened: Kubernetes stopped an old pod, but the ALB still sent it requests for a few seconds (502 Bad Gateway, timeouts). New pods also started receiving traffic as soon as Kubernetes considered them ready, before the ALB health check had confirmed them.

The fix: three parts

Part	Where	Effect
Pod readiness gates	namespace label <code>elbv2.k8s.aws/pod-readiness-gate-inject: enabled</code> (Terraform)	the controller adds a condition to new pods: they count as <i>ready</i> only when the ALB reports the target healthy, so the rollout waits for the ALB
preStop sleep 15 s	<code>lifecycle.preStop.sleep.seconds: 15</code> in the Deployment	a terminating pod keeps serving for 15 s while the ALB removes it
Deregistration delay 10 s	Ingress annotation <code>deregistration_delay.timeout_seconds=10</code>	the ALB stops sending new requests to a removed target after 10 s, inside the 15 s window

`terminationGracePeriodSeconds: 30` gives the pod enough time for the 15 s pause plus nginx's shutdown.

```
terminationGracePeriodSeconds: 30
containers:
  - name: web
    lifecycle:
      preStop:
        sleep: {seconds: 15}
```

The `sleep` action is built into Kubernetes, so the image needs no `sleep` binary.

After the fix, the same test during another deployment:

```
seconds: 211    200 OK: 401                                → 401 of 401 requests succeeded
```

You can see the readiness gate on the new pods:

```
profile-card-68fbcbf49d-jf9zb  gates=target-health.elbv2.k8s.aws/k8s-profilec-profilec-37d81badd9
```

Version skew

The measurement showed something else: for about **28 seconds**, the answers alternated between the old JavaScript bundle and the new one. Old and new pods were serving side by side, which is normal in a rolling update.

For a single-page app this can cause **version skew**: a browser loads the old `index.html` from an old pod, then asks a new pod for the old `index-xTPJYkoq.js`, which the new pod does not have (404). Common solutions: keep old asset files available for a while (for example on S3 or a CDN), or route a user's requests to one version (sticky sessions).

Try it

During a deployment, in a second terminal:

```
H=<ALB hostname>
while true; do curl -s -o /dev/null -m 3 -w '%{http_code}\n' "http://$H/"; sleep 0.5; done | sort | uniq -c
```

Stop it with Ctrl+C after the rollout and count the non-200 answers.

Common mistakes

- Trusting `maxUnavailable: 0` alone behind an external load balancer.
- A `preStop` sleep shorter than the deregistration delay (or no `preStop` at all).
- A `terminationGracePeriodSeconds` shorter than the `preStop` sleep: the pod is killed mid-sleep.
- Measuring zero downtime with one manual refresh instead of steady traffic.

Check yourself

1. Why did requests fail during the first measured deployment?
2. What does a pod readiness gate wait for?
3. Why must the `preStop` sleep be longer than the deregistration delay?

Answers

▼ Answers

1. Old pods stopped while the ALB still sent them requests, and new pods could get traffic before the ALB had confirmed them healthy.

2. For the ALB target health check: the pod counts as ready only when the load balancer reports it healthy.
3. So the pod keeps serving until the ALB has stopped sending it new requests; if the pod stops first, those requests fail.

13. Teardown and cost

What is it?

Teardown means deleting everything a project created. **Verification** means proving it, instead of assuming it.

Why it matters

- A forgotten lab costs about \$7 per day here, and much more with bigger clusters.
- In a shared account, leftovers confuse other people and can block their work (quotas, overlapping names).
- "I ran destroy" is not proof: some resources are created outside Terraform, and some are global.

How it works

The order matters

`scripts/teardown.sh` :

1. **Delete the Ingress** and wait until the ALB `eks-devsecops-profile-card` no longer exists. The ALB, its target groups and security groups were created by the controller, not by Terraform. If Terraform deleted the controller first, they would be left behind, and the VPC could not be deleted.
2. `terraform destroy` .
3. `scripts/verify-teardown.sh` .

Before starting, switch the pipeline off (`gh variable set DEPLOY_ENABLED --body false`) so a push cannot deploy into a cluster that is being deleted.

Recorded:

```
==> Deleting the Ingress (the controller removes the load balancer)
==> Load balancer deleted
module.vpc.aws_nat_gateway.this[0]: Destruction complete after 1m20s
module.eks.aws_eks_cluster.this[0]: Destruction complete after 3m29s
Destroy complete! Resources: 72 destroyed.
```

The verification script

`scripts/verify-teardown.sh` only **reads**. It checks, in eu-central-1: EKS cluster, VPC, subnets, NAT gateways, Elastic IPs, EC2 instances, network interfaces, security groups, the load balancer and its target groups, the ECR repository, CloudWatch log groups, the KMS alias, and anything else tagged `Project=eks-devsecops` . Then the global IAM roles and policies, and finally that the **shared GitHub OIDC provider still exists**.

Two lessons from the real run made the script better:

1. **Module-generated names**. The EKS and VPC modules named some IAM roles `default-eks-node-group-...` and `vpc-flow-log-role-...` , without the project prefix. A check for `eks-devsecops*` would have missed them. Now the script checks those prefixes too, but counts a role or policy only if it carries the

tag `Project=eks-devsecops`, so another team's similarly named roles never cause a false alarm.

Terraform now also gives every IAM name the `eks-devsecops` prefix.

2. **The tagging index lags.** Right after the destroy, the Resource Groups Tagging API still listed 4 tagged resources: a NAT gateway (state `deleted`), a flow log and a security group (both already gone), and the KMS key. The script now checks each tagged item's **real state**.

The **KMS key** is special: AWS never deletes a key immediately. It goes to `PendingDeletion` for at least 7 days (here: deleted on 2026-10-11). A key waiting for deletion is not billed and cannot be used.

Final recorded result:

```
gone EKS cluster ... gone anything else tagged Project=eks-devsecops
gone IAM roles    gone IAM policies
kept GitHub OIDC identity provider
Clean: nothing of this project is left.
```

And the region inventory taken before the project matched exactly afterwards: 1 VPC (the default one), 0 Elastic IPs, 0 NAT gateways, 0 clusters, 0 load balancers, 0 ECR repositories, 0 instances.

What it cost

Item	Approximate price in eu-central-1
EKS control plane	\$0.10 / hour
2 × t3.medium	\$0.10 / hour
NAT gateway	\$0.05 / hour + data
ALB, public IPv4, logs, KMS	~\$0.05 / hour

From the first `terraform apply` to the verified teardown: **62 minutes**, so an estimated **\$0.35**. These are on-demand list prices; the AWS bill (Cost Explorer, usually within 24 hours) is the final word.

Try it

```
gh variable set DEPLOY_ENABLED --body false
export TF_VAR_allowed_account_id=<your account id>
scripts/teardown.sh
scripts/verify-teardown.sh          # safe to run any time, read-only
gh secret delete AWS_DEPLOY_ROLE_ARN --env production
```

Common mistakes

- `terraform destroy` before deleting the Ingress (the VPC deletion fails).
- Checking only one region, or forgetting IAM.
- Trusting the tagging API without checking each resource's real state.
- Deleting resources by name pattern in a shared account (you may hit someone else's).
- Forgetting the GitHub secret and variable after the cloud side is gone.

Check yourself

1. Why is the Ingress deleted before `terraform destroy` ?
2. Why does the verification check tags as well as name prefixes for IAM?
3. The KMS key still exists after the teardown. Is that a leftover?

Answers

▼ Answers

1. The ALB and its security groups were created by the Load Balancer Controller. Deleting the Ingress lets the controller remove them; otherwise they stay behind and block the VPC deletion.
2. Name prefixes find the candidates (including module-generated names); the tag proves they belong to this project, so other teams' roles with similar names are never reported or touched.
3. No. AWS requires a waiting period of at least 7 days before deleting a key; it is in `PendingDeletion`, cannot be used, and is not billed.

14. Hands-on labs

Nine labs. **Labs 1–4 are free** and run on your computer with Docker. **Labs 5–9 run in your own AWS account and cost money** (roughly \$0.30 per hour while the platform exists).

 **Before lab 5:** use your **own** AWS account, never an employer's account without permission. Plan the AWS labs for one sitting, and **always finish with lab 9 (teardown) the same day**.

All commands run from the repository root, in bash (Git Bash on Windows). Compare what you see with [docs/test-results.md](#).

Lab 1: Build and harden the image locally (free)

```
docker build -t profile-card:local --build-arg VERSION=0.1.0 --build-arg COMMIT=abcdef1 app
docker run -d --name pc --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges -p
18080:8080 profile-card:local
curl -sI localhost:18080/ ; curl -s localhost:18080/healthz ; docker exec pc id
docker rm -f pc
```

1. Which user does nginx run as? Which security headers do you see?
2. Remove `--tmpfs /tmp` and start it again. What happens, and why does the Kubernetes Deployment mount an `emptyDir` at `/tmp`?
3. `cd app && npm ci && npm test`: which 5 behaviours do the tests check?

Lab 2: Be the vulnerability gate (free)

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.75.0 \
image --severity CRITICAL,HIGH --ignore-unfixed profile-card:local
```

1. Comment out the `RUN apk upgrade --no-cache` line (and the `USER 0` above it) in `app/Dockerfile`, rebuild, scan again. Is anything found today? If yes, which package and which fixed version?
2. Scan once without `--ignore-unfixed`. How many more findings appear, and why does the pipeline not block on them?
3. Undo your change: `git checkout -- app/Dockerfile`.

Lab 3: Break the Dockerfile, let the checks catch it (free)

```
docker run --rm -i hadolint/hadolint:v2.15.1 < app/Dockerfile # clean
printf '\nUSER root\n' >> app/Dockerfile
docker run --rm -i hadolint/hadolint:v2.15.1 < app/Dockerfile # which rule fires?
git checkout -- app/Dockerfile
```

1. Which hadolint rule fails, and what does it say?
2. In `k8s/deployment.yaml`, which setting would still make the pod run as UID 10101, even with `USER root` in the image?

Lab 4: Read the security exceptions and the manifests (free)

```
docker run --rm -v "$PWD:/src" -w /src aquasec/trivy:0.75.0 \
  config --severity HIGH,CRITICAL --skip-dirs terraform/.terraform terraform      # without the
ignore file
docker run --rm -v "$PWD:/src" -w /src aquasec/trivy:0.75.0 \
  config --severity HIGH,CRITICAL --ignorefile .trivyignore.yaml --skip-dirs terraform/.terraform
terraform
```

(Run `terraform -chdir=terraform init -backend=false` first so the modules are downloaded.)

1. Which three findings appear without the ignore file? For each, write down the production fix from `.trivyignore.yaml`.
2. Change one `expired_at` to yesterday's date and scan again. What happens? Why is that useful?
3. Render the manifests like CI does (see `.github/workflows/ci.yaml`, job `infrastructure`) and run `kubeconform -strict` on them.

Lab 5: Prepare your fork and read your OIDC subject (free, needs GitHub)

1. Fork the repository. In `terraform/variables.tf`, set `github_repository` to your fork.
2. Read your subject prefix and set `github_oidc_subject_prefix`: `bash gh api repos/<you>/<your-fork>/actions/oidc/customization/sub` Does your prefix contain numeric IDs (`owner@id/name@id`)? What attack does that prevent?
3. Create the GitHub environment `production` that only allows the `main` branch (Settings → Environments → Deployment branches).
4. If your AWS account has **no** GitHub OIDC provider yet (there is at most one per account, `token.actions.githubusercontent.com`), set `create_github_oidc_provider = true` so Terraform creates it. Leave it `false` (the default) in an account that already has one: the provider is shared by every repository there, and this project then only reads it.

Lab 6: Plan, review, apply (💰 costs money from here)

```
export AWS_REGION=eu-central-1
export TF_VAR_allowed_account_id=<your 12-digit account id>
aws ec2 describe-vpcs --query 'length(Vpcs)'      # write down your inventory first
terraform -chdir=terraform init
terraform -chdir=terraform plan -out=tfplan
terraform -chdir=terraform apply tfplan
```

1. What does the last line of the plan say? What would make you stop before applying?
2. Try a wrong account ID in `TF_VAR_allowed_account_id` and run `plan`. What happens?
3. After the apply: `aws eks update-kubeconfig --name eks-devsecops` and `kubectl get nodes -o wide`. Is there an external IP?

Lab 7: Deploy through the pipeline and check least privilege (💰)

```
terraform -chdir=terraform output -raw github_deploy_role_arn | gh secret set AWS_DEPLOY_ROLE_ARN --env production
gh variable set DEPLOY_ENABLED --body true
gh workflow run deploy.yaml --ref main
```

1. Open the run. How long did each job take? Which URL does it show?
2. Find the "Least privilege check" step. Which `can-i` answers are `no`?
3. Make a small visible change in `app/src/profile.js`, push to `main`, and watch the footer version change on the live site.

Lab 8: Attack your cluster (🔪)

```
kubectl -n profile-card run attacker --image=busybox:1.37 --privileged --restart=Never -- sleep 60
P=$(kubectl -n profile-card get pods -o jsonpath='{.items[0].metadata.name}')
kubectl -n profile-card exec "$P" -- wget -T 5 -q -O /dev/null http://example.com
kubectl run imds --image=curlimages/curl:8.17.0 --restart=Never --rm -i --command -- \
  curl -sS -m 5 -X PUT http://169.254.169.254/latest/api/token -H "X-aws-ec2-metadata-token-ttl-seconds: 60"
IP=$(kubectl -n profile-card get pods -o jsonpath='{.items[0].status.podIP}')
kubectl -n default run lateral --image=curlimages/curl:8.17.0 --restart=Never --rm -i -- curl -m 5
"http://$IP:8080/"
```

1. Which mechanism stops each of the four attempts?
2. Temporarily change the NetworkPolicy's `ipBlock` to `10.42.0.0/16` (`kubectl -n profile-card edit networkpolicy allow-http-from-load-balancer`) and repeat the lateral test. What changes? Put it back (`kubectl apply -k` or re-run the pipeline).
3. While a deployment runs (Lab 7, step 3), run the `curl` loop from chapter 12 and count non-200 answers.

Lab 9: Teardown and proof (do not skip)

```
gh variable set DEPLOY_ENABLED --body false
scripts/teardown.sh
scripts/verify-teardown.sh
gh secret delete AWS_DEPLOY_ROLE_ARN --env production
aws ec2 describe-vpcs --query 'length(Vpcs)' # same as your inventory from lab 6?
```

1. What did the script delete first, and why?
2. Does the tagging API still list anything right after the destroy? What does the script report for it?
3. What state is the KMS key in, and when will AWS remove it?
4. Next day: open Cost Explorer and compare the real cost with the estimate (about \$0.30 per hour).

Glossary

Term	Meaning
access entry (EKS)	an EKS API object that gives an IAM principal Kubernetes permissions, optionally scoped to namespaces; replaces the <code>aws-auth</code> ConfigMap
actionlint	a linter for GitHub Actions workflow files
add-on (EKS)	an AWS-provided cluster component: VPC CNI, CoreDNS, kube-proxy, Pod Identity agent
ALB	Application Load Balancer: AWS's managed HTTP/HTTPS load balancer
AWS Load Balancer Controller	a Kubernetes controller that creates ALBs from Ingress objects
availability zone (AZ)	an isolated data center (or group) inside an AWS region
CIDR	notation for an IP range, e.g. <code>10.42.0.0/16</code>
control plane	the Kubernetes API server, scheduler and database; run by AWS in EKS
cosign	Sigstore's tool to sign and verify container images
CSP	Content-Security-Policy: a response header that tells the browser which scripts and styles it may run
CVE	the public ID of a known vulnerability
default deny	a NetworkPolicy that blocks all traffic, so only explicit exceptions are allowed
deregistration delay	how long an ALB keeps a removed target before forgetting it
digest	the SHA-256 hash of an image (<code>sha256:...</code>); identifies exact content and cannot be moved
ECR	Elastic Container Registry: AWS's image registry
EKS	Elastic Kubernetes Service: Kubernetes with an AWS-managed control plane
emptyDir	a temporary, pod-local directory
environment (GitHub)	a named deployment target (<code>production</code>) with rules, e.g. only the <code>main</code> branch
gitleaks	a scanner for secrets in git history
hadolint	a linter for Dockerfiles
Helm	a package manager for Kubernetes; Terraform uses it to install the load balancer controller
hop limit (IMDS)	how many network hops the IMDSv2 token response may travel; 1 keeps it away from pods
IAM	Identity and Access Management: users, roles and policies in AWS
IAM role	an identity without a password whose temporary credentials can be assumed
immutable subject	GitHub's OIDC subject format with numeric owner and repository IDs
immutable tags (ECR)	an image tag cannot be overwritten once pushed
IMDS / IMDSv2	the EC2 instance metadata service; v2 requires a session token

Term	Meaning
Ingress	a Kubernetes object describing HTTP routing from outside into Services
internet gateway	a VPC's connection to the internet
IRSA	IAM Roles for Service Accounts: the older way to give pods AWS permissions (not used here)
KMS	Key Management Service: AWS's managed encryption keys
kubeconform	validates Kubernetes manifests against the official schemas
Kustomize	builds Kubernetes manifests from a base and edits (here: the image digest)
lateral movement	an attacker moving from one compromised system to another
least privilege	every identity gets only the permissions it needs
managed node group	EC2 worker nodes whose lifecycle EKS manages
multi-stage build	a Dockerfile that builds in one stage and copies only the result into a small final stage
NAT gateway	lets private subnets start outbound connections; nothing can connect in
NetworkPolicy	Kubernetes rules for which traffic pods may receive and send
OIDC	OpenID Connect: identity with signed tokens; GitHub uses it so jobs can assume AWS roles
Pod Identity (EKS)	gives a Kubernetes service account an IAM role without keys
Pod Security Admission	built-in check that enforces <code>privileged</code> , <code>baseline</code> or <code>restricted</code> levels per namespace
preStop hook	an action Kubernetes runs before stopping a container (here: sleep 15 s)
private / public subnet	a subnet without / with a route from the internet gateway
readiness gate	an extra condition a pod must meet before it counts as ready (here: ALB target healthy)
region	a geographic area of AWS, e.g. <code>eu-central-1</code> (Frankfurt)
rolling update	replacing pods one by one during a deployment
route table	rules that send a subnet's traffic to a gateway
SBOM	software bill of materials: the list of components in an image (Syft, CycloneDX)
seccomp	a Linux filter that blocks dangerous system calls (<code>RuntimeDefault</code>)
security group	an AWS firewall around network interfaces
securityContext	Kubernetes settings for a pod's or container's privileges
smoke test	a quick check that the deployed system answers
state (Terraform)	Terraform's record of the resources it manages
target type ip	the ALB sends traffic directly to pod IP addresses
trust policy	the part of an IAM role that says who may assume it
Trivy	a scanner for vulnerabilities and misconfigurations
version skew	old and new versions of an app running at the same time and mixing requests
VPC	Virtual Private Cloud: your private network in AWS

Term	Meaning
VPC CNI	the EKS network plugin; gives pods VPC IPs and enforces NetworkPolicies
VPC flow logs	records of network connections in a VPC
zizmor	a security auditor for GitHub Actions workflows

Interview questions

25 questions this project prepares you for. Try to answer before opening the answer.

1. Why put EKS worker nodes in private subnets, and how do they still pull images?

▼ Answer

Nodes without public IPs cannot be reached from the internet, which removes a whole attack surface. Outbound traffic (ECR, EKS API, OS updates) goes through a NAT gateway in a public subnet; only the load balancer is public.

2. One NAT gateway or one per availability zone?

▼ Answer

One is cheaper (this lab), but it is a single point of failure: if its zone fails, the other zone loses outbound access. Production uses one per AZ, each private subnet routing to the NAT in its own zone.

3. What guard rails make Terraform safe in a shared AWS account?

▼ Answer

`allowed_account_ids` (refuses any other account), a validated region variable, default tags on every resource, reviewing the plan ("0 to change, 0 to destroy"), reading shared resources with data sources instead of managing them, and unique name prefixes.

4. Why must Terraform state never be committed, and where should a team keep it?

▼ Answer

It contains resource IDs, the account number and sometimes secrets, and two copies diverge. Teams use a remote backend such as S3 with locking so only one apply runs at a time.

5. What are EKS access entries and why are they better than the aws-auth ConfigMap?

▼ Answer

They map IAM principals to Kubernetes permissions through the EKS API, as code, with AWS-managed access policies that can be scoped to namespaces. The ConfigMap was edited by hand inside the cluster and a mistake could lock everyone out.

6. How does IMDSv2 with a hop limit of 1 protect a cluster?

▼ Answer

The instance metadata service hands out the node's AWS credentials. IMDSv2 requires a session token from a PUT request; with a hop limit of 1 the token response cannot reach a pod (one extra hop), so a

compromised pod cannot steal the node's credentials. In this project the token request from a pod timed out and IMDSv1 returned 401.

7. Why encrypt Kubernetes secrets with a KMS key when EBS volumes are already encrypted?

▼ Answer

Envelope encryption protects secrets in etcd and backups at the application level with a key you control and can audit or revoke, independent of disk encryption.

8. What is Pod Identity and why use it instead of IRSA?

▼ Answer

Pod Identity links a service account to an IAM role through an EKS association and an agent add-on. Unlike IRSA it needs no IAM OIDC provider per cluster and no role annotation, so fewer global IAM objects are created.

9. How does GitHub Actions get into AWS without stored keys?

▼ Answer

The job requests an OIDC token from GitHub, `configure-aws-credentials` exchanges it with STS (`AssumeRoleWithWebIdentity`) for temporary credentials of a role whose trust policy checks the audience (`sts.amazonaws.com`) and the subject (this repository's `production` environment).

10. The OIDC role assumption fails with "Not authorized to perform sts:AssumeRoleWithWebIdentity". How do you debug it?

▼ Answer

Compare the token's claims with the trust policy: audience, and especially the subject. New GitHub repositories use an immutable subject with numeric IDs (`repo:owner@id/name@id:environment:production`); read it with `gh api repos/OWNER/REPO/actions/oidc/customization/sub`. Also check that the job runs in the expected environment and that the identity provider exists.

11. Why scope the trust policy to a GitHub environment instead of a branch?

▼ Answer

An environment can have protection rules (only `main`, optional reviewers) and secrets of its own. Only jobs that pass those rules get the `environment:production` subject.

12. How do you prove a pipeline has least privilege in Kubernetes?

▼ Answer

Scope its access entry to one namespace and test it on every run with `kubectl auth can-i : yes` for its own namespace, no for kube-system, secrets, cluster role bindings and namespace deletion. The job fails

if an answer changes.

13. Why split the deploy workflow into a build-and-scan job and a deploy job?

▼ Answer

Scanning tools never run next to cloud credentials, a vulnerable image never reaches the registry, and the deploy job ships exactly the scanned image (passed as an artifact).

14. Why deploy by digest instead of tag?

▼ Answer

A tag can be moved to different content; a digest is the content hash. The pipeline writes the digest into the kustomization, so pods run exactly what was scanned and signed. ECR tags are also immutable.

15. What does `--ignore-unfixed` do in a Trivy gate and why use it?

▼ Answer

It reports only vulnerabilities with a fixed version available, so the gate blocks on problems the team can act on. A gate that blocks on unfixable issues gets disabled.

16. The scanner blocks the official base image you pinned yesterday. What do you do?

▼ Answer

Read the finding: here pcre2 CVE-2026-103111 (HIGH, fixed in 10.49). If the distribution has the fix, apply it in the image (`apk upgrade` as root, then back to the unprivileged user) or move to a newer base digest; rescan to confirm 0 findings. Never just ignore it.

17. How do you handle a scanner finding you decide to accept?

▼ Answer

Write it down where the scanner reads it (`.trivyignore.yaml`) with the reason, the production fix and an expiry date, so it is reviewed again. Here: the public EKS endpoint for GitHub-hosted runners and node egress.

18. Why pin GitHub Actions to commit SHAs?

▼ Answer

Version tags can be moved to malicious commits (it happened to tj-actions/changed-files in 2025). A full commit SHA always means the same code. A script in CI rejects unpinned actions.

19. How does an Ingress become an AWS load balancer?

▼ Answer

The AWS Load Balancer Controller watches Ingresses with `ingressClassName: alb`, finds subnets by their tags, and creates an ALB, listeners, target groups and security groups. With target type ip, targets are the pod IPs.

20. **Why does `terraform destroy` fail on the VPC if you forget the Ingress?**

▼ Answer

The ALB and its security groups were created by the controller, not Terraform. If the controller is destroyed first, they remain and still use the VPC's subnets. Delete the Ingress first and wait for the ALB to disappear.

21. **On EKS, a `NetworkPolicy` allows the VPC CIDR so the ALB can reach the pods. What is wrong?**

▼ Answer

With the VPC CNI, pod IPs come from the VPC, so the rule also lets every pod in the cluster reach the app (lateral movement; measured: HTTP 200 from another namespace). Allow only the subnets where the ALB's interfaces live (here 10.42.48.0/23).

22. **Which `securityContext` settings make a container hard to abuse?**

▼ Answer

`runAsNonRoot` with a high UID, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem: true` (with a small writable `/tmp`), `capabilities: drop [ALL]`, `seccompProfile: RuntimeDefault`, no service account token, and resource limits; enforced by Pod Security `restricted` on the namespace.

23. **Why can a rolling update behind an ALB still drop requests, and how do you fix it?**

▼ Answer

Pods stop before the ALB stops sending them traffic, and new pods can get traffic before the ALB has health-checked them. Fix: pod readiness gates (pods are ready only when the ALB target is healthy), a `preStop` sleep longer than the deregistration delay (15 s vs 10 s), and enough termination grace. Measured: 3 of 280 failed before, 401 of 401 after.

24. **What is version skew in a single-page app rollout?**

▼ Answer

Old and new pods serve at the same time (about 28 s here), so a browser can load an old `index.html` and then ask a new pod for an old hashed asset that no longer exists. Keep old assets available (S3/CDN) or keep users on one version.

25. **How do you prove a lab is fully deleted in a shared account?**

▼ Answer

Delete in the right order, then verify read-only: every resource type by name and by project tag in the region, IAM roles and policies (including module-generated names, confirmed by tag), the real state of everything the tagging API still lists (it lags; a KMS key in its 7-day deletion window counts as deleted), and that shared resources such as the GitHub OIDC provider still exist. Finally compare with the inventory taken before the project.